past plays. Most of the time we will play greedy, but sometimes we will take a risk and choose a random lever to see what happens. The result will, of course, influence our future greedy actions. Let's see if we can solve this in code with Python.

Listing 2.2. Epsilon-greedy strategy for action selection

```
import numpy as np
from scipy import stats
import random
import matplotlib.pyplot as plt

n = 10                          1
probs = np.random.rand(n)       2
eps = 0.2                       3
```

- ***1*** Number of arms (number of slot machines)
- ***2*** Hidden probabilities associated with each arm
- ***3*** Epsilon for epsilon-greedy action selection

In this casino example, we will be solving a 10-armed bandit problem, so *n* = 10. We've also defined a numpy array of length *n* filled with random floats that can be understood as probabilities. Each position in the `probs` array corresponds to an arm, which is a possible action. For example, the first element has index position 0, so action 0 is arm 0. Each arm has an associated probability that weights how much reward it pays out.

The way we've chosen to implement our reward probability distributions for each arm is this: Each arm will have a probability, e.g., 0.7, and the maximum reward is $10. We will set up a `for` loop going to `10`, and at each step it will add `1` to the reward if a random float is less than the arm's probability. Thus, on the first loop it makes up a random float (e.g., 0.4). 0.4 is less than 0.7, so `reward += 1`. On the next iteration, it makes up another random float (e.g., 0.6) which is also less than 0.7, so `reward += 1`. This continues until we complete 10 iterations, and then we return the final total reward, which could be anything between 0 and 10. With an arm probability of 0.7, the *average* reward of doing this to infinity would be 7, but on any single play it could be more or less.

Listing 2.3. Defining the reward function